

TOPSoccer Membership Reports — Build Plan

Modernizing the legacy `MASTER.xlsm` (Excel + VBA macros) into Google Sheets + Apps Script, driven off the live GotSport "raw data" export.

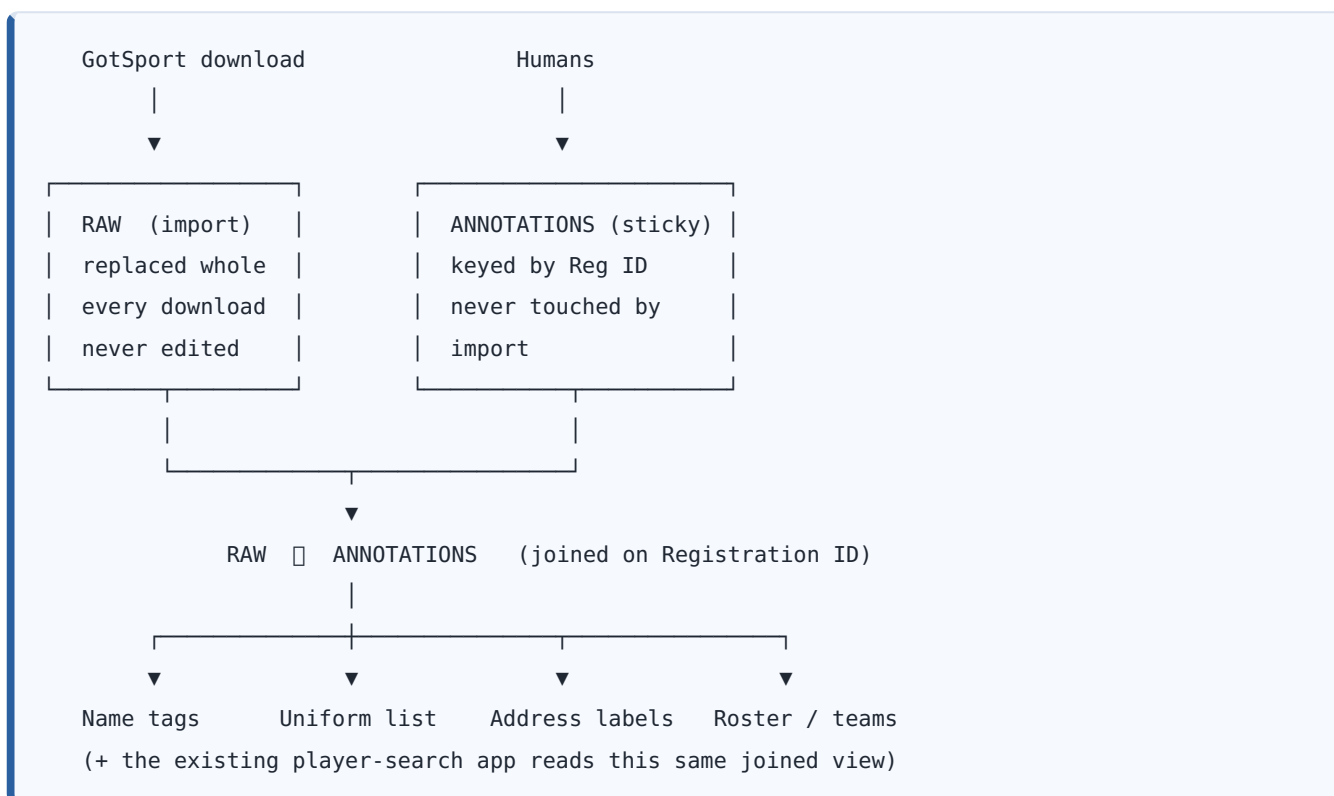
Date: 2026-06-27 · **Target account:** SDTSAdmin@gmail.com

Confirmed decisions

- **Source of truth = the GotSport *Program Registrations* export** — re-downloaded frequently (especially early season). Carries the custom form fields (incl. T-Shirt Size); the player-list export does not.
 - **Data is dynamic:** late arrivals, team reassignments, and other changes appear with each new download. Re-imports must be cheap and safe.
 - **Sticky info that must survive every re-import:** free-text note per record, "shirt picked up," manual team override, and the legacy extras (banquet count, Fall Classic, consent flags).
 - **Reports to carry over:** name tags, uniform lists, address labels, roster / team sorting.
 - **Delivery = both:** a web app (self-service, no edit access — for the volunteers who just print) **and** an in-sheet Apps Script menu (for whoever manages the data).
 - Everything lives under the **SDTSAdmin** org account.
-

The core idea — three layers

The whole design exists to solve one tension: *raw data gets overwritten often, but human-added info must persist*. We separate the two and join them on a stable key.



1. **Raw** — a verbatim dump of the latest GotSport export. Replaced wholesale on each download. Never hand-edited.
2. **Annotations** — keyed by **Registration ID**. Holds everything people add. A re-import never touches it.
3. **Reports** — computed live from the joined data, so they always reflect the newest download.

The raw feed + stable key

Raw feed = the GotSport *Program Registrations* export (Programs → Programs List → submitted count → Download as CSV), pulled per program (Player + Coach). Validated against a real export: it carries everything we need — **Registration ID, T-Shirt Size (143/144 filled, 11 sizes), guardian addresses, contacts, age group**. The *player-list* export is **not** used (it lacks shirt size, and its IDs are a different namespace).

Stable key = Registration ID. Confirmed **unique** (144/144) and **stable across re-downloads** (per Dean) — so it's the sole sticky key; no name+DOB fallback needed.

Dynamics handled by this design:

- *Late arrivals* — show up in Raw on the next download; reports include them automatically; a blank annotation row is created for them on demand.
- *Team reassignments* — handled by the Team Override + Source-of-Record dial (see Team reassignment workflow). GotSport ships blank, so our sheet is the default team source.
- *Sticky notes & flags* — persist by Registration ID no matter how many times Raw is replaced.

Coaches export separately. The validation pull returned players only (the coach program didn't come through in the same file). Coaches are needed for name tags + the coach roster, so the importer expects a **separate coach export** (or confirm how GotSport bundles them).

Tab schemas (draft)

Raw_Players / **Raw_Coaches** (or one **Raw** tab with a **Type** column) — the paste/import target, exact GotSport columns. Untouched except by import.

Annotations — one row per Registration ID, created/maintained by people and scripts:

Column	Source	Notes
Registration ID	join key	from Raw
Type	Player / Coach	
Note	manual	the "special sticky note"
Shirt Picked Up	app / manual	timestamp; migrated from the player-search app
Team Override	manual	blank = use Raw's team
Banquet Count	manual	legacy extra
Fall Classic	manual	legacy extra (X)
Consent / Pix / Park Release	manual	legacy flags
Last Updated / Updated By	auto	audit

Reports never store data — they're regenerated from **Raw** + **Annotations** each run.

Important: migrate the player-search app's write-back

Your existing **player-search web app** currently writes **"Shirt Picked Up"** and **"Registration Note"** directly **onto the registration sheet rows**. The first time you re-import raw data over that sheet, those entries get wiped.

Fix (no extra system — same Annotations layer): repoint the app's `markShirt()` and `saveNote()` to write into **Annotations**, keyed by Registration ID, and have its search read **Raw** + **Annotations**. This protects the app from re-imports *and* makes its notes/pickups the same sticky store the reports read. One shared layer serves both tools.

The reports

The legacy MASTER produced **plain sorted lists**, not label-stock layouts (confirmed by inspecting the file — no print areas or label templates). The modern version generates true **Avery label/badge layouts** as print-ready PDFs.

- **Name tags** — adhesive badges (default **Avery 5395**, 8 per sheet, 2½" × 3¾"): player/coach name prominent with team below, sorted by team order then name.
- **Uniform list** — First / Last / Team / **Shirt size** (T-shirt only; shorts & socks are not tracked). Source: GotSport's T-Shirt Size column.
- **Address labels** — Full Name / Street / City, State ZIP, built from the guardian address fields in Raw, sorted (default **Avery 5160**, 30 per sheet, 1" × 2⅝").
- **Roster / team sorting** — teams in custom order then name; plus a Team Assignments view (First / Last / Team / Shirt / Paid / Comments).
- **Pending Team Changes** (*admin/Jess*) — every player where Team Override ≠ raw GotSport team; her worklist for updating GotSport (see Team reassignment workflow).

Custom team order (fixed, consistent year to year): Hotwheels, Dribblers, Shooters, Wings, Kickers, Passers, Strikers, Sweepers. Hardcoded as the sort order; validated against the Team values in each import.

Report sort order — config default + runtime override

Sort order is **not locked now**; it's handled the same way as the label picker. Each report has a **config-backed default sort**, and whoever runs it can **pick a different sort at request time**. Admins change a default in Config without recoding.

Seed defaults (from the proven legacy macros):

Report	Default sort
Roster (Master)	Last, then First
Coaches	Team order (Hotwheels...Sweepers, <i>Staff</i> last), then Title (Head Coach → Co-Head → Assistant → Floater), then Last, First
Name tags	Team order, then Last, First
Uniforms	Team order, then Last, First
Address labels	Last, First (alternate: ZIP, for bulk mail)

Open decisions to revisit with real early-season data in hand (apply across reports, captured as config choices — no rebuild needed):

- **Group-by-team vs. flat alphabetical** — by-team gives each coach a clean stack; alphabetical suits a central check-in table. Both can be offered via the runtime picker.
- **Last-name vs. first-name primary** — youth programs sometimes sort by first name; decide per report.

- **Where unassigned players go** — early season many have no team yet. Options: a visible "Unassigned" group at the top (recommended — it's the worklist someone's clearing), bottom, or omitted. Defined as a config choice.

Label format chosen at request time

The label engine is **geometry-driven**, not tied to one stock. A small **template registry** holds each Avery spec — page size, columns × rows, label width/height, top/left margins, and gutters. Adding a new stock is one row of numbers. When someone requests labels or name tags (web-app button or sheet menu), they pick the template from a **dropdown**, with 5160 (address) and 5395 (name tags) pre-selected as defaults. The generator renders the chosen geometry on the fly — so format is a runtime choice, never a code change.

Team reassignment workflow

Team moves often happen on the fly (game day). This rides on the **Team Override** annotation already in the design — no new data layer — plus a coach-facing action and one admin report. It replaces text/email churn with a single live worklist.

Lifecycle: behavior follows the **Teams Source of Record** config dial.

1. A coach sets a player's team override in the web app — records who, when, and (optional) why. *Assigning an unassigned player is the same action* (override from blank → team), so this also clears the early-season "Unassigned" worklist.
2. Reports reflect the change **immediately** (override wins) — useful day-of.
3. If **Teams source** = **sheets** (today's default — GotSport teams blank): the override *is* the assignment. Our sheet is the system of record; the Pending Team Changes report is informational only. If **Teams source** = **gotSPORT** (once Jess ramps up): the override is a **proposal**. Jess runs the **Pending Team Changes** report (every player where Override ≠ raw GotSport team) and applies each in GotSport Roster Builder.
4. In **gotSPORT** mode, the next raw download arrives with the team matching → the override **auto-retires** (marks *applied*). Overrides never accumulate.

GotSport *does* support admin team assignment (Roster Builder: drag, or bulk "Add to Team") and team-selection at registration — so the "push to GotSport" path is real, not blocked. The only choice is whether Jess's worklist ends at our sheet (**sheets** mode) or continues into Roster Builder (**gotSPORT** mode). Flipping the config is the entire migration.

Access — who can reassign (default: **coach passcode**). A shared code gates the reassign action (same pattern as the Photo Manager's **ADMIN_KEY**). Coaches commit directly so it's instant on the field; Jess's report is the safety net since she reviews every change before it becomes official in GotSport. Alternatives considered: open-to-all (too loose) and Jess-only with coach requests (safest but re-adds back-and-forth).

Annotation fields used: Team Override, plus override audit (Changed By / Changed At / Reason / Status = pending|applied).

Delivery

- **Web app** — extend the existing player-search app with a **Reports** section: each report is a button that generates a print-ready PDF from current data. Same URL/QR, runs as SDTSAdmin, anyone with a Google account, no edit access. This is what the "different people on their own schedule" crowd uses.
 - **In-sheet menu** — an Apps Script custom menu for the data manager: *Import Latest Download*, *Rebuild Reports*, *Find Duplicates*, etc. (mirrors how your expense-tracker reconciler menu works).
-

Import workflow — drop-folder pickup

A dedicated **"GotSport Drop" Drive folder** under SDTSAdmin. The data manager downloads the export from GotSport and drops the file in. An Apps Script (menu item *Import Latest Download*, optionally a scheduled trigger) picks up the newest file, parses it, and **replaces the Raw tab**. Annotations are untouched; new Registration IDs get blank annotation rows; the imported file is moved to an **Imported** subfolder (dedupe by filename) — mirroring the reconciler's work-area pattern.

Implementation detail to confirm at build time: the exact file GotSport delivers — one workbook with separate player and coach tabs, or two separate files (and CSV vs. XLSX). The importer will handle both gracefully.

Resolved specifics (2026-06-27)

1. **Raw feed = Program Registrations export** (not the player list); validated to carry Registration ID + T-Shirt Size + addresses.
 2. **Registration ID is stable + unique** → the sole sticky key (no name+DOB fallback needed).
 3. **Uniform = T-shirt only** (no shorts/socks); shirt size confirmed present (11 sizes).
 4. **Team names are fixed** year to year: Hotwheels, Dribblers, Shooters, Wings, Kickers, Passers, Strikers, Sweepers.
 5. **Label stock chosen at request time** from a template registry; defaults 5160 (address) / 5395 (name tags).
 6. **Import = drop-folder pickup** (see Import workflow above).
 7. **Config is its own dedicated Sheets file** — separate from the website config (same pattern, no connection).
 8. **Team source of record is a config dial** — **sheets** today, flip to **gotsport** when Jess ramps up.
 9. **Coaches export separately** from players (to be confirmed how GotSport bundles them).
-

Configuration (its own dedicated config file — *separate from the website*)

Driven by config wherever it reduces future recoding — the **same pattern** as the website, but **explicitly NOT the same file**. The membership system gets its **own dedicated Google Sheets config file**, with zero connection to the website's Config sheet: no shared tabs, no cross-references, no dependency in either

direction. They evolve independently. (Mechanically, the Apps Script reads this config by its own sheet ID, exactly as the site reads its separate Config sheet.)

A small set of config tabs holds the things most likely to change, so an admin edits a row instead of editing code:

- **GotSport column map** (*highest value*) — `internal field` → `GotSport header text`. The importer resolves columns through this map, so when GotSport renames/reorders/adds export columns, you fix a row, not code. Formalizes the flexible matching the player-search app already does.
- **Source of Record** (*see table below*) — per data domain, whether the value comes from GotSport (Raw) or from our sheet (Annotations). This is the dial that lets Jess move GotSport authority forward on her own timeline.
- **Label template registry** — one row per Avery stock (number, page size, cols × rows, label W/H, top/left margin, gutters). The runtime label picker reads this; new stock = new row.
- **Operational settings** — drop-folder ID, Raw/Annotations tab names + sheet IDs, default label template per report.
- **Team list + sort order** — Hotwheels...Sweepers as a config list (fixed today, but a future add/rename never touches code).
- **Annotation field definitions** — which sticky fields exist (note, shirt pickup, team override, banquet, Fall Classic, consent). Add/retire a tracked field via config.
- **Report defaults** — per report: title, default label template, sort key. (*Full per-column layout stays in code — config there costs more than it saves.*)

Source of Record — the team-authority dial

GotSport currently holds **no team data** (blank in every export — TOPSoccer has always assigned teams outside GotSport). So the system ships with teams sourced from our sheet, and flips to GotSport whenever Jess is ready — by changing **one config value**, no rebuild:

Domain	Source (default)	Notes
Teams	sheets → gotsport	flip to <code>gotsport</code> when Jess ramps up Roster Builder; the override-staging logic then turns the Pending Team Changes report into her active worklist
Contact info	gotsport	always from registration
Shirt size	gotsport	from the T-Shirt Size field
Notes / flags / pickup	sheets	sticky-only, never in GotSport

This table also *is* the rule for what's a sticky Annotation field vs. read from Raw. Teams is wired up first (the live case); the other rows are ready as Jess expands GotSport's role.

Build phases

1. **Scaffolding** — create Raw + Annotations + **Config** tabs, seed Config (column map, team list, label registry, settings), and the Registration ID join.
2. **Import** — script + menu to replace Raw safely (resolving columns via the config map) and lazily create annotation rows.
3. **App migration** — move player-search shirt/note write-back into Annotations.
4. **Report generators** — the four reports + Pending Team Changes, as print-ready PDFs, reading the config-driven label registry.
5. **Web app Reports section** — buttons wired to the generators, with the runtime label-template picker.
6. **Team reassignment** — passcode-gated coach action (set Team Override) + the Pending Team Changes report + auto-retire on raw catch-up.

Each phase is independently testable, and nothing here touches your live registration data until you say go.